```
"""
data_cleaning.py

Cleans the raw Nassau Candy Distributor
shipment data and prepares it for
the SQL and Power BI steps. Roughly what
happens, in order:

1. Load the raw CSV
2. Check structure, nulls, dtypes
3. Fix date formats (source dates come in
as DD-MM-YYYY strings)
4. Check the Ship Date column for how
usable it actually is, then either
   use it directly or fall back to a
documented estimate
5. Clean up the geographic text fields
6. Map each product to the factory that
makes it, so routes can be built
7. Add the route and lead time features
8. Write out the cleaned CSV

Each step is its own function so you can
run pieces individually and
look at the output in between if something
looks off.
"""

import pandas as pd
```

```
RAW_PATH =
"data/Nassau_Candy_Distributor.csv"
CLEAN_PATH =
"data/Nassau_Candy_Distributor_clean.csv"

# Used only to check how bad the Ship Date
problem is, see
# calculate_lead_time() below. A shipment
taking longer than 30 days
# to leave the factory isn't realistic for
this kind of business.
MAX_PLAUSIBLE_LEAD_DAYS = 30

# Ship Mode to an estimated typical lead
time in days. This is used as a
# fallback when Ship Date turns out not to
be usable. In this dataset,
# literally every row (all 10,194) fell
outside a plausible 0 to 30 day
# range, with only 28 distinct values in
the whole column, so dropping
# the bad rows wasn't an option since there
were no good ones left.
# These numbers are placeholders based on
typical real-world shipping
# tiers. Swap in your own measured averages
if you ever get a trustworthy
# Ship Date column to work with.
SHIP_MODE_LEAD_TIME = {
    "Standard Class": 4.97,
    "Second Class": 3.02,
    "First Class": 2.01,
    "Same Day": 0.52,
}
```

```
# Product name to factory, taken from the
project's product/factory
# reference table. Two products below
weren't in that source table and
# got assigned by pattern instead, worth
double checking against the
# full reference table before treating
those two as settled.
PRODUCT_TO_FACTORY = {
    "Wonka Bar - Nutty Crunch Surprise":
"Lot's O' Nuts",
    "Wonka Bar - Fudge Mallows": "Lot's O'
Nuts",
    "Wonka Bar -Scrumdiddlyumptious":
"Lot's O' Nuts",
    "Wonka Bar - Milk Chocolate": "Wicked
Choccy's",
    "Wonka Bar - Triple Dazzle Caramel":
"Wicked Choccy's",  # not in source table,
assumed
    "SweeTARTS": "Sugar Shack",
    "Nerds": "Sugar Shack",
    "Fun Dip": "Sugar Shack",
    "Fizzy Lifting Drinks": "Sugar Shack",
    "Everlasting Gobstopper": "Secret
Factory",
    "Lickable Wallpaper": "Secret Factory",
    "Wonka Gum": "Secret Factory",
    "Hair Toffee": "The Other Factory",
    "Kazookles": "The Other Factory",
    "Laffy Taffy": "Sugar Shack",  # not in
source table, assumed
}
```

```
def load_raw_data(path: str) ->
pd.DataFrame:
    """Load the raw CSV into a
DataFrame."""
    df = pd.read_csv(path)
    print(f"Loaded {len(df):,} rows and
{df.shape[1]} columns from {path}")
    return df


def inspect_data(df: pd.DataFrame) -> None:
    """Quick look at the data before
touching anything."""
    print("\ndtypes:")
    print(df.dtypes)

    print("\nnull counts:")
    print(df.isnull().sum())

    print(f"\n{df.duplicated().sum()} exact
duplicate rows found")


def fix_date_columns(df: pd.DataFrame) ->
pd.DataFrame:
    """Convert Order Date and Ship Date
from DD-MM-YYYY strings into real
    datetime objects so date math works on
them."""
    df["Order Date"] =
pd.to_datetime(df["Order Date"],
format="%d-%m-%Y", errors="coerce")
```

```
    df["Ship Date"] =
pd.to_datetime(df["Ship Date"], format="%d-
%m-%Y", errors="coerce")

    bad_order_dates = df["Order
Date"].isna().sum()
    bad_ship_dates = df["Ship
Date"].isna().sum()
    if bad_order_dates or bad_ship_dates:
        print(f"Warning: {bad_order_dates}
Order Dates and {bad_ship_dates} "
              f"Ship Dates couldn't be
parsed")

    return df


def calculate_lead_time(df: pd.DataFrame) -
> pd.DataFrame:
    """Check how usable Ship Date actually
is, then decide what to do
    about it. Don't assume the column is
fine or broken, measure it first."""
    raw_lead = (df["Ship Date"] - df["Order
Date"]).dt.days
    invalid_mask = raw_lead.isna() |
(raw_lead < 0) | (raw_lead >
MAX_PLAUSIBLE_LEAD_DAYS)
    invalid_pct = invalid_mask.mean()

    print(f"\nShip Date check:
{invalid_mask.sum():,} of {len(df):,} rows
"
          f"({invalid_pct:.1%}) fall
```

```
outside a plausible 0 to "
          f"{MAX_PLAUSIBLE_LEAD_DAYS} day
range. Distinct raw values: "
          f"{raw_lead.nunique()}.")

    if invalid_pct > 0.5:
        # More than half the column is bad,
dropping those rows would gut
        # the dataset rather than clean it,
so fall back to the Ship Mode
        # based estimate instead.
        print("Ship Date isn't reliable for
most rows here, so using the "
              "Ship Mode based lead time
estimate instead. See "
              "SHIP_MODE_LEAD_TIME at the
top of this file.")
        df["Shipping_Lead_Time_Days"] =
df["Ship Mode"].map(SHIP_MODE_LEAD_TIME)
        unmapped =
df["Shipping_Lead_Time_Days"].isna().sum()
        if unmapped:
            print(f"Warning: {unmapped}
rows have a Ship Mode not covered "
                  "by SHIP_MODE_LEAD_TIME,
extend the mapping to cover them.")
        return df
    else:
        # The column is trustworthy enough
to use as is. Flag and remove
        # the minority of bad rows, and
keep a copy of what got removed.
        df["Shipping_Lead_Time_Days"] =
raw_lead
```

```
        invalid_rows =
df[invalid_mask].copy()
        clean_rows =
df[~invalid_mask].copy()
        if len(invalid_rows) > 0:

invalid_rows.to_csv("data/removed_invalid_l
ead_times.csv", index=False)
            print(f"Removed
{len(invalid_rows):,} invalid rows, saved
to "

"data/removed_invalid_lead_times.csv for
reference.")
        return clean_rows


def standardize_geography(df: pd.DataFrame)
-> pd.DataFrame:
    """Clean up text fields so grouping and
joining doesn't quietly break
    because of extra spaces or inconsistent
casing."""
    text_cols = ["City", "State/Province",
"Country/Region", "Region", "Division"]
    for col in text_cols:
        df[col] =
df[col].astype(str).str.strip().str.title()

    # Ship Mode and Product Name need to
keep their original casing
    # ("First Class" etc), so just strip
whitespace on those.
    df["Ship Mode"] = df["Ship
```

```
Mode"].astype(str).str.strip()
    df["Product Name"] = df["Product
Name"].astype(str).str.strip()

    return df


def assign_factory(df: pd.DataFrame) ->
pd.DataFrame:
    """Look up the manufacturing factory
for each product, needed to build
    factory to customer routes."""
    df["Factory"] = df["Product
Name"].map(PRODUCT_TO_FACTORY).fillna("Unma
pped")

    unmapped = (df["Factory"] ==
"Unmapped").sum()
    if unmapped:
        print(f"\nNote: {unmapped} rows
couldn't be mapped to a factory, "
              "add any missing product
names to PRODUCT_TO_FACTORY.")

    return df


def engineer_route_features(df:
pd.DataFrame) -> pd.DataFrame:
    """Build the route and month fields
used throughout the analysis."""
    df["Route"] = df["Factory"] + " -> " +
df["State/Province"]
    df["Order_Month"] = df["Order
```

```
Date"].dt.to_period("M").astype(str)
    return df


def export_clean_data(df: pd.DataFrame,
path: str) -> None:
    """Write the cleaned dataset out to
disk."""
    df.to_csv(path, index=False)
    print(f"\nSaved cleaned dataset:
{len(df):,} rows to {path}")


def main():
    df = load_raw_data(RAW_PATH)
    inspect_data(df)
    df = fix_date_columns(df)
    df = calculate_lead_time(df)
    df = standardize_geography(df)
    df = assign_factory(df)
    df = engineer_route_features(df)
    export_clean_data(df, CLEAN_PATH)


if __name__ == "__main__":
    main()
```